



서울시립대학교

Lec 4 - Clock & Timer

전력전자연구실 교수 이 승 환



✓ 다음과 같은 clock source들이 사용된다.

- 1) HSI(High Speed Internal) :
 - 내부 8 MHz RC oscillator clock, system reset 이후에 기본적으로 선택된다.
- 2) HSE(High Speed External) :
 - MCU 외부에서 4[MHz]~32[MHz] clock을 OSC 또는 Crystal 소자를 이용하여 공급.
- 3) PLL(Phased Locked Loop) :
 - PLL를 이용하여 HSI 또는 HSE clock을 채배 해 사용한다.
- 4) LSI(Low Speed Internal) RC clock :
 - 독립적인 watchdog를 구동하고, 선택적으로 Stop 또는 Standby mode로부터 Auto-wakeup을 위해 사용되는 RTC를 구동하는 40[KHz].
- 5) LSE(Low Speed External) :
 - 선택적으로 realtime clock(RTCCLK)를 구동하는 32.768[kHz] crystal과 같은 외부 소자에서 공급.

Clock 설정 방법 - 2

RCC Mode and Configuration

Mode

High Speed Clock (HSE) Crystal/Ceramic Resonator

Low Speed Clock (LSE) Crystal/Ceramic Resonator

☒ Master Clock Output

☒ Audio Clock Input (I2S_CKIN)

Configuration

Reset Configuration

☒ Parameter Settings ☒ User Constants ☒ NVIC Settings ☒ GPIO Settings

Configure the below parameters :

Search (Ctrl+F)

System Parameters

VDD voltage (V) 3.3 V

Prefetch Buffer Enabled

Flash Latency(WS) 2 WS (3 CPU cycle)

RCC Parameters

HSI Calibration Value 16

HSE Startup Timeout Value (ms) 100

LSE Startup Timeout Value (ms) 5000

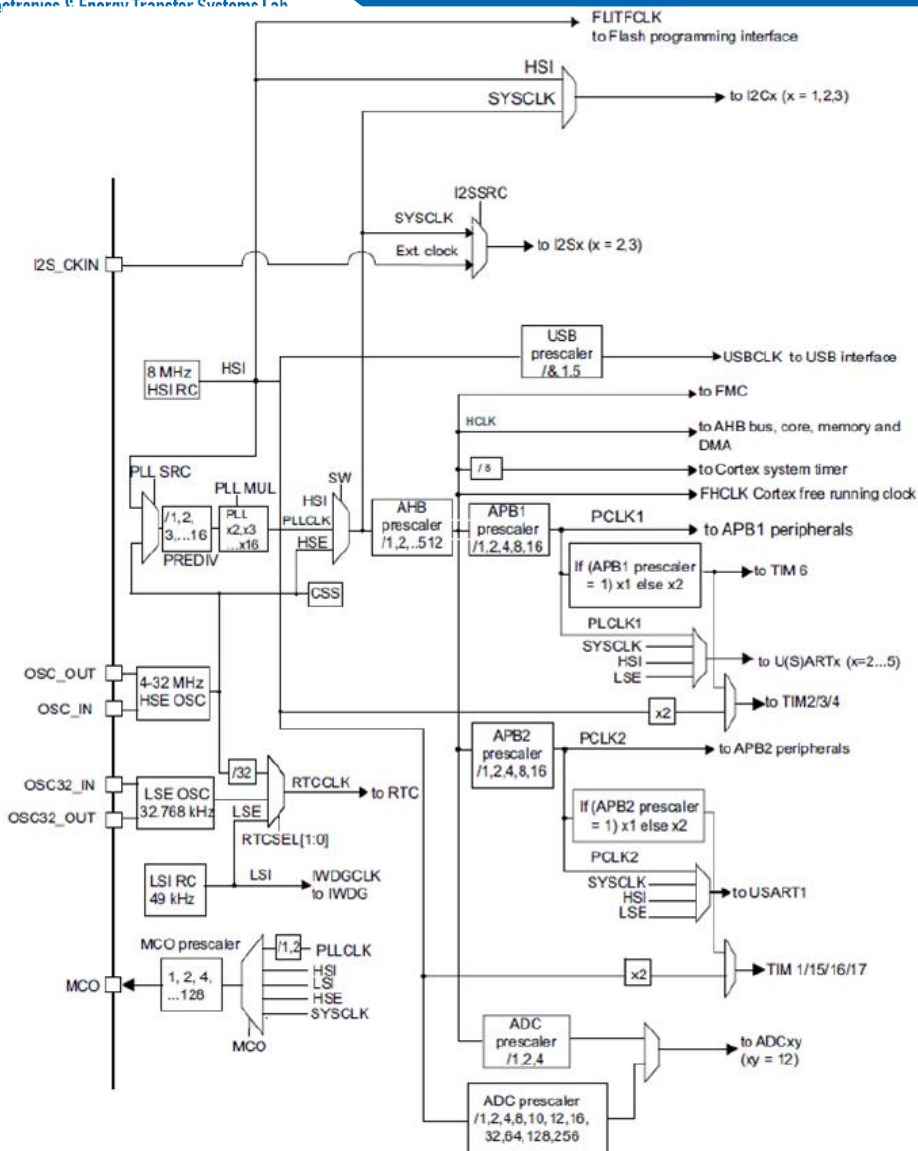
LSE Drive Capability LSE oscillator high drive capability

LSE oscillator low drive capability

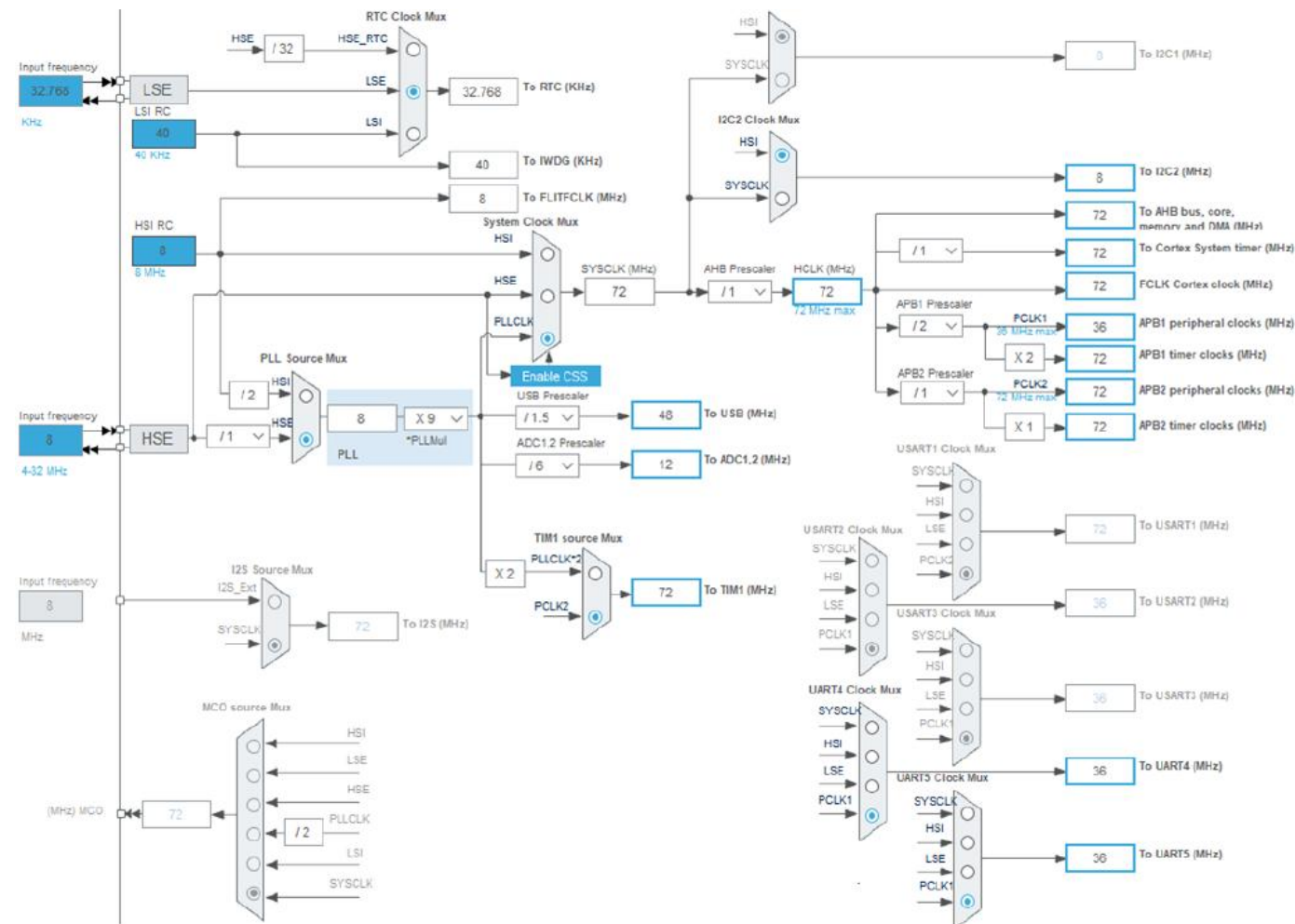
LSE oscillator medium low drive capability

LSE oscillator medium high drive capability

LSE oscillator high drive capability



(a) STM32F302xD/E clock tree

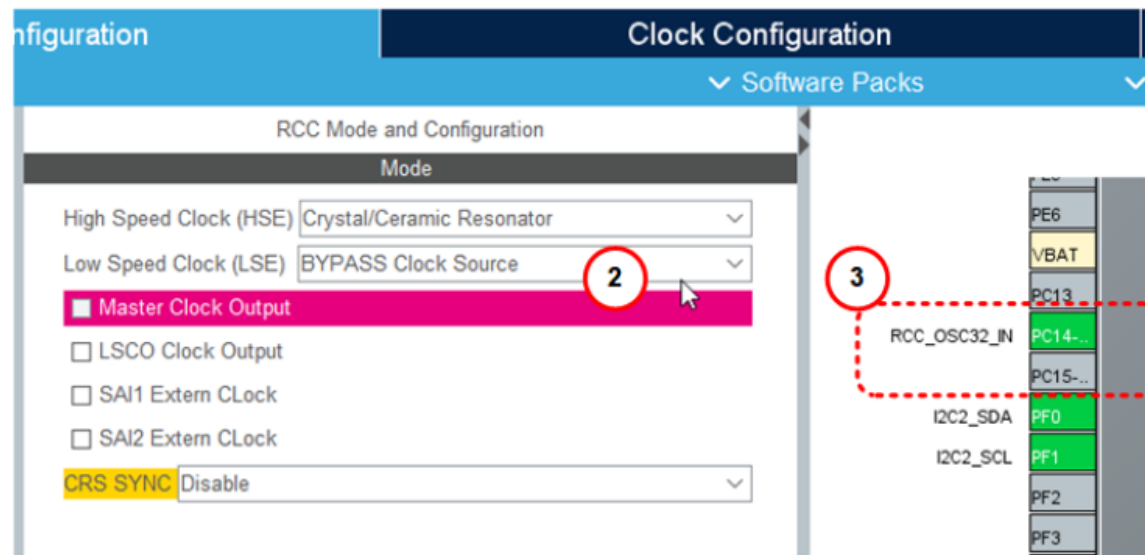
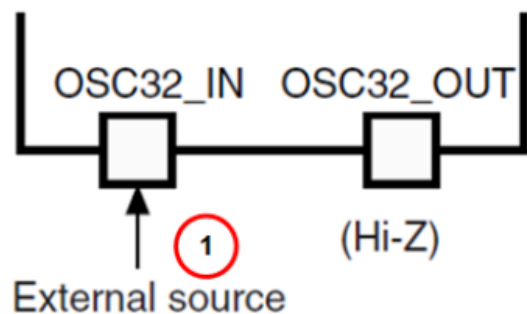


(b) CubeMX Clock Configuration for STM32F302xD/E clock tree

✓ OSC32_IN/OUT 연결 방법 :

- 1) 외부에서 OSC 또는 어떠한 방법으로 32.768[kHz] clock을 정확히 제공할 수 있다면, ①번에서 보여준 것과 같이 OSC32_IN에 연결하고, OSC32_OUT은 NC 처리해 주면 된다.
- 2) ②번과 같이 CubeMX에서 BYPASS Clock Source를 선택하여 주어야 한다. 그러면, ③번과 같이 OSC32_OUT pin(즉, PC15)이 floating으로 바뀐 것을 볼 수 있다.
- 3) Crystal 또는 Ceramic resonator를 이용하는 경우에는 OSC32_IN으로 공급하고, OSC32_OUT으로 내보내 주어야 한다.

Hardware configuration



✓ Timer의 개념 :

- 지정한 일정한 시간 간격마다 interrupt를 발생시킴.

✓ Timer의 용도 :

- “Cortex-M Interrupt Vector Table - 1”에서 보여준 NVIC default interrupt들 중에 15번째 interrupt가 SysTick이다.
- 이것은 system timer로서 OS를 사용하는 경우에 일정한 그리고, 아주 짧은 시간 간격을 생성하기 위해서 사용되며, Cortex-M core에 내장되어 있는 기본 timer이다.
- Hal_Delay(); 함수에서 SysTick timer 사용하여 1[ms] 단위 delay 발생.

✓ Basic Timers : TIM6과 TIM7 :

- 동작 중에 조정 가능한 prescaler에 의해서 구동되는 16bits auto-reload counter로 구성.
- 입력과 출력 pins가 없다.
- 용 도 :
 - ① 일반적인 시간 간격 설정.
 - ② 다른 timer의 master로 사용.
 - ③ DAC를 위한 time 구간 설정 - Chapter 14.에서 자세히 설명.

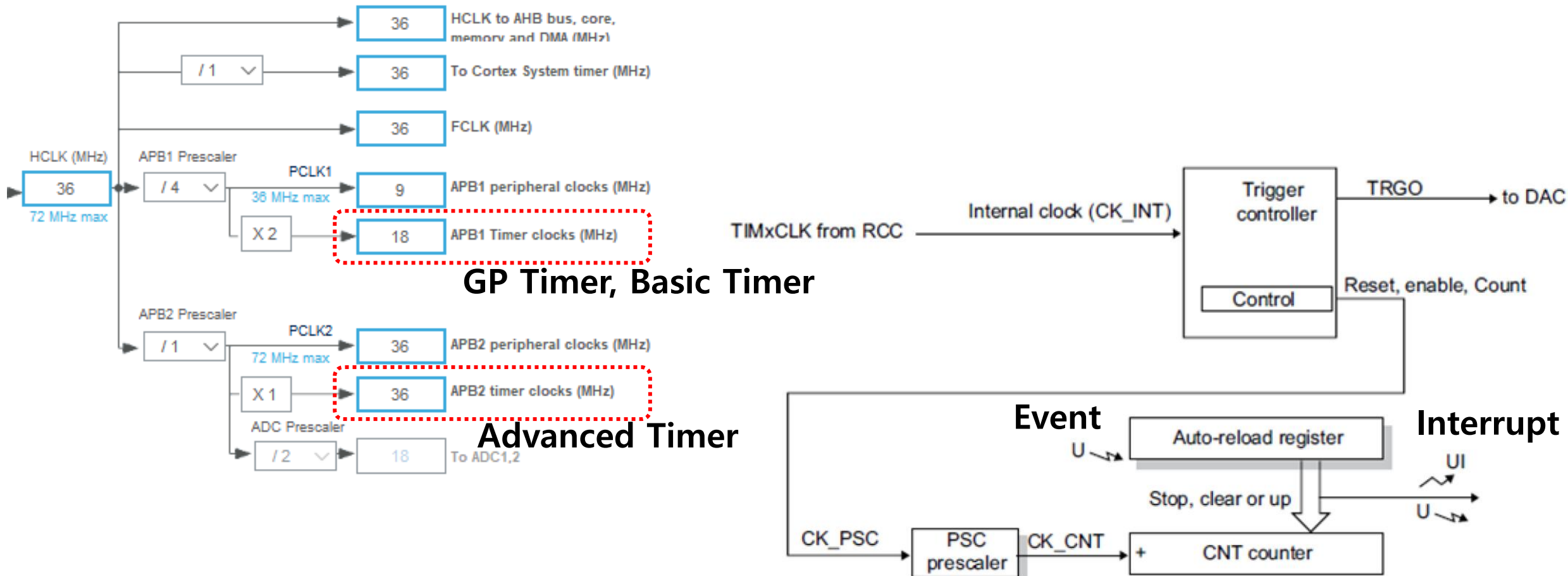
- ✓ General Purpose(GP) timers : TIM2~TIM5, TIMy
 - 동작 중에 조정 가능한 prescaler에 의해서 구동되는 16bits auto-reload counter로 구성.
 - STM32F1xx MCU를 제외한 나머지 상위 MCU는 32bits auto-reload counter도 내장.
 - Timer 3, Timer 4는 16bits auto-reload counter, Timer 2와 Timer 5는 32bits auto-reload counter.
 - basic timer 기능들을 모두 수행.
 - 최대 4개까지의 PWM channel들 구성이 가능.
 - GP timer와 Advanced control timers는 함께 동기화될 수 있다. 구체적으로 외부 신호로 timer를 동기화할 수 있고, 여러 timers를 함께 상호 연결하여 동기화할 수도 있다.
 - 다음과 같은 interrupt 생성
 - - Update Interrupt : Counter Overflow/Underflow 발생, counter 초기화.
 - - Trigger Interrupt : Counter Start, Stop, 초기화 또는 내부/외부 trigger에 의한 count.
 - - 입력 capture와 출력 비교.

- ✓ Advanced control timers : TIM1과 TIM8, TIMx
 - 동작 중에 조정 가능한 prescaler에 의해서 구동되는 16bits auto-reload counter로 구성.
 - 모든 GP timer 기능을 수행.
 - timer의 출력 신호를 reset 상태 또는 알려진 상태로 설정하는 break 입력 기능.
 - 추가적으로 모터 제어와 전력 변환에 관계된 기능들을 제공,
 - deadtime 설정 기능.
 - 180도 위상차 PWM 생성 기능.
- ✓ 몇몇 MCU는 하나의 master와 5개의 slave timer들 즉, 6개 timer들로 고정밀의 시간 간격을 생성할 수 있는 High Resolution timers와 저전력용 low power timer를 제공한다.

✓ Timer 종류별 비교 :

Timer	Counter resolution	Counter type	Prescaler factor	DMA request generation	Capture/compare channels	Complementary outputs
TIM1, TIM8	16-bit	Up, down, up/down	Any integer between 1 and 65536	Yes	4	Yes
TIM2, TIM3, TIM4, TIM5	16-bit	Up, down, up/down	Any integer between 1 and 65536	Yes	4	No
TIM6, TIM7	16-bit	Up	Any integer between 1 and 65536	Yes	0	No

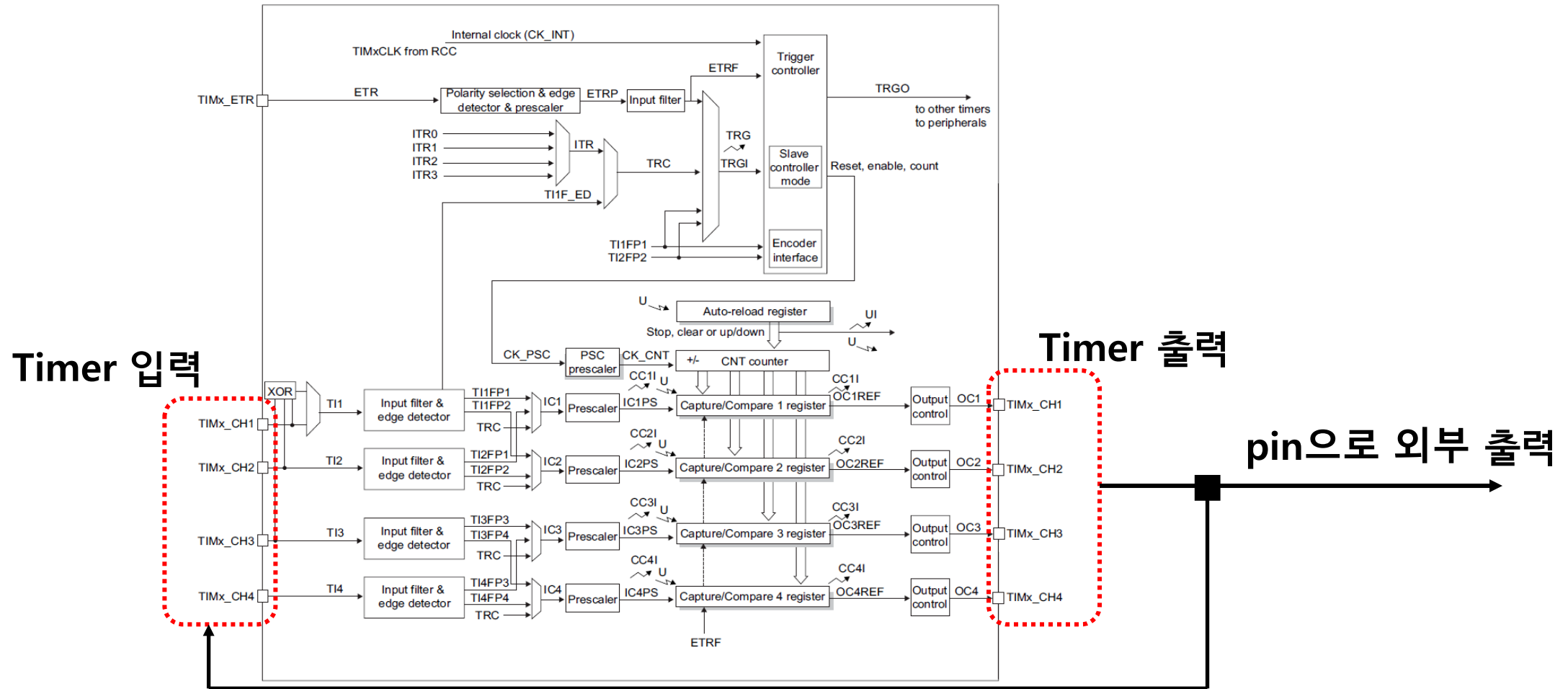
Timer의 종류와 내부 구조 - 1.



(a) Basic Timer 내부 구조.

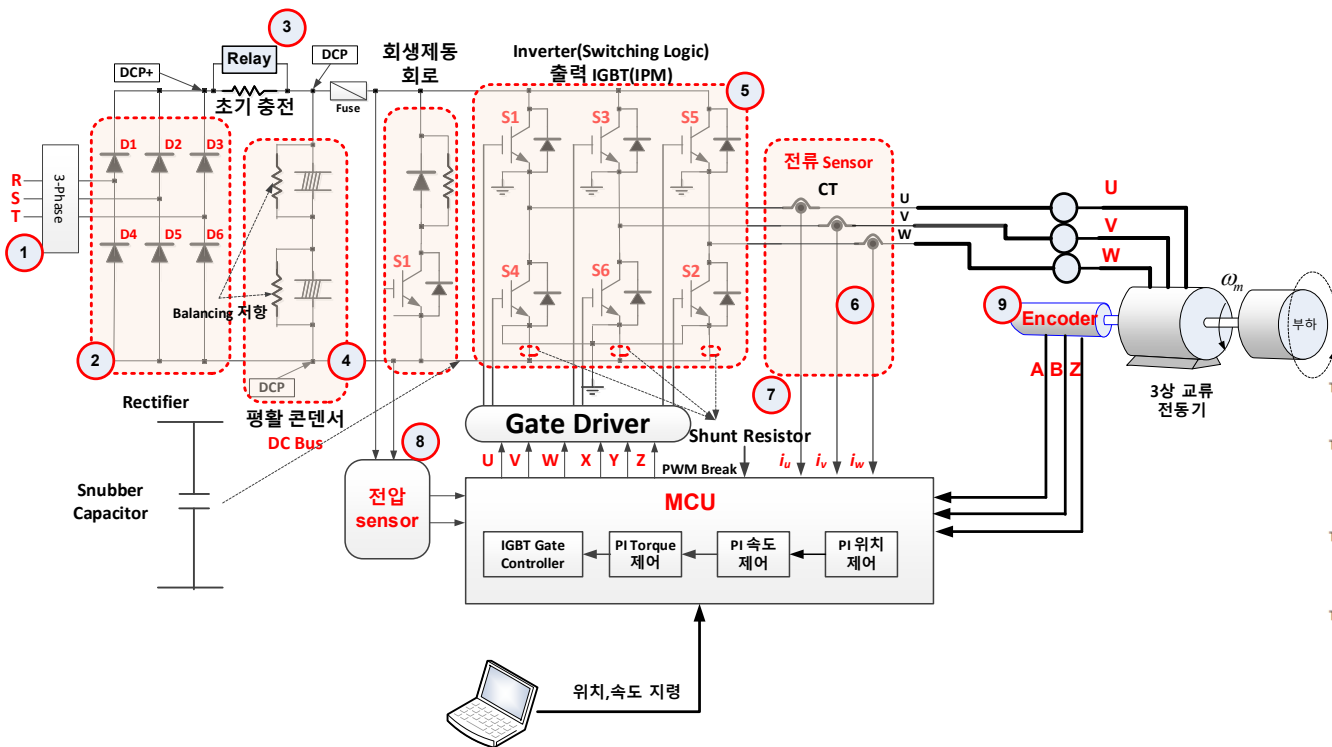
Timer의 종류와 내부 구조 - 2.

- ✓ General Purpose(GP) timers 내부 구조 :

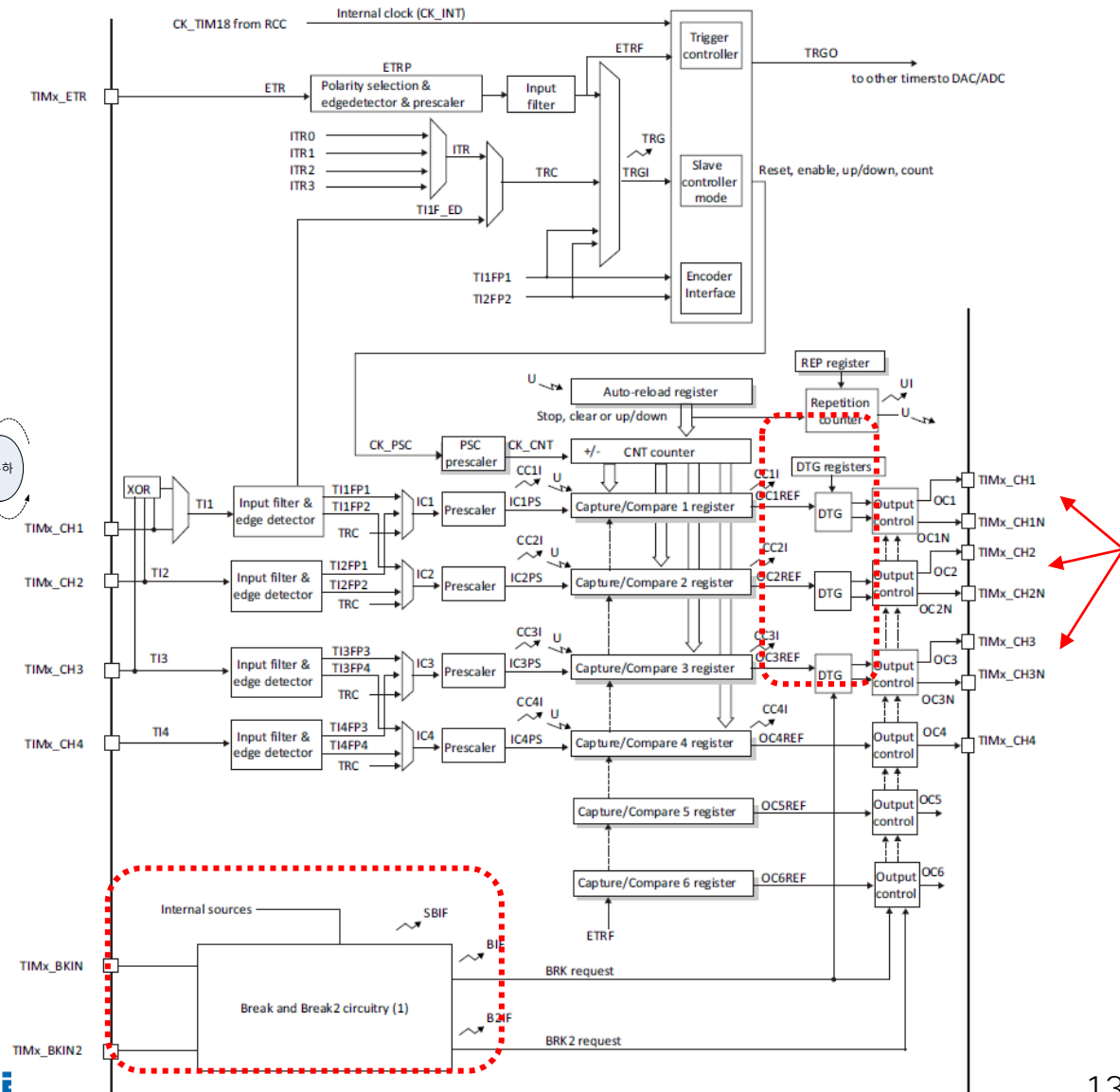


Timer의 종류와 내부 구조 - 3.

✓ Advanced control timers 내부 구조 :

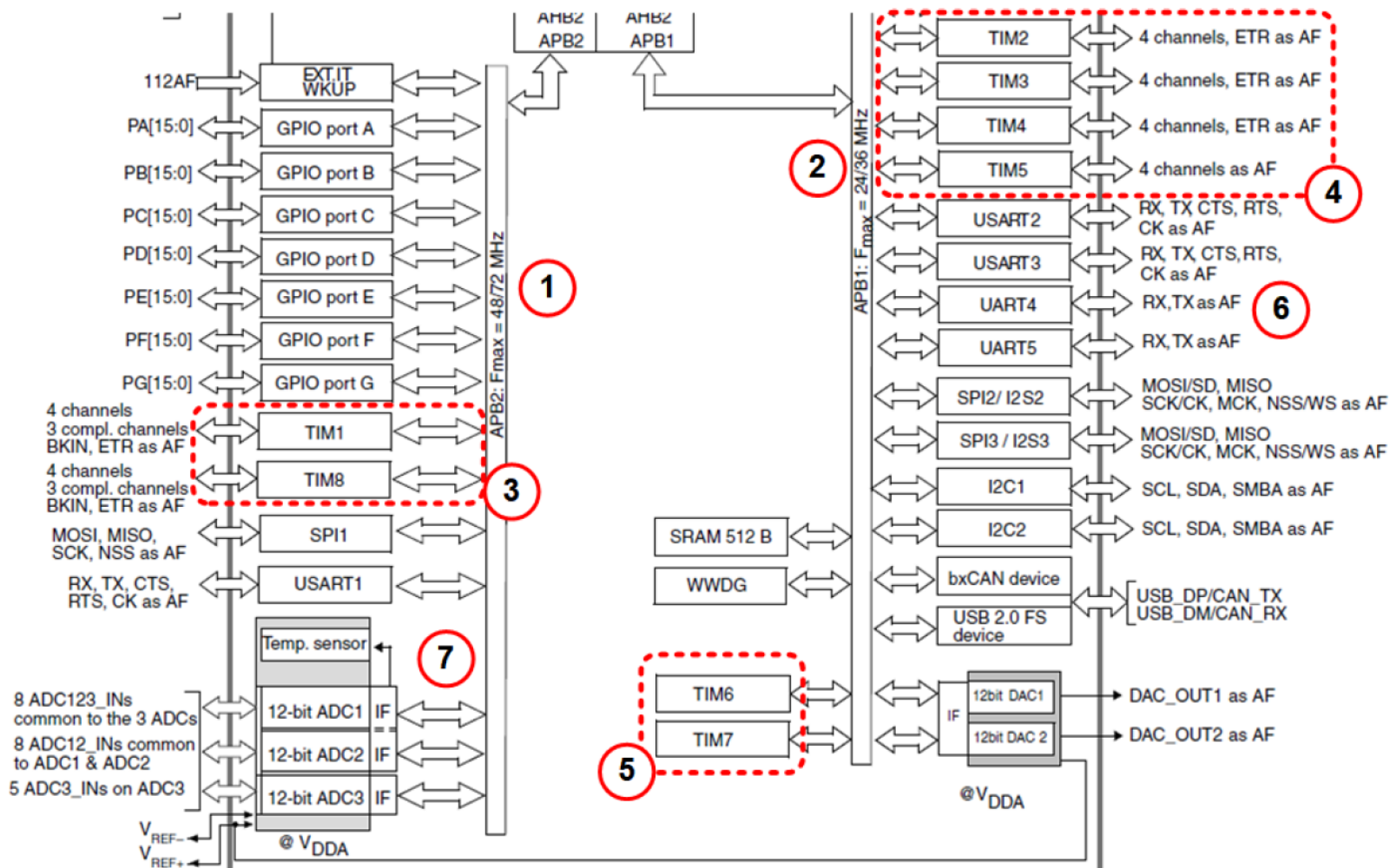


3상 교류 전동기 전체 구조.



Timer clock tree - 1.

✓ STM32F103 내부 구성도 :



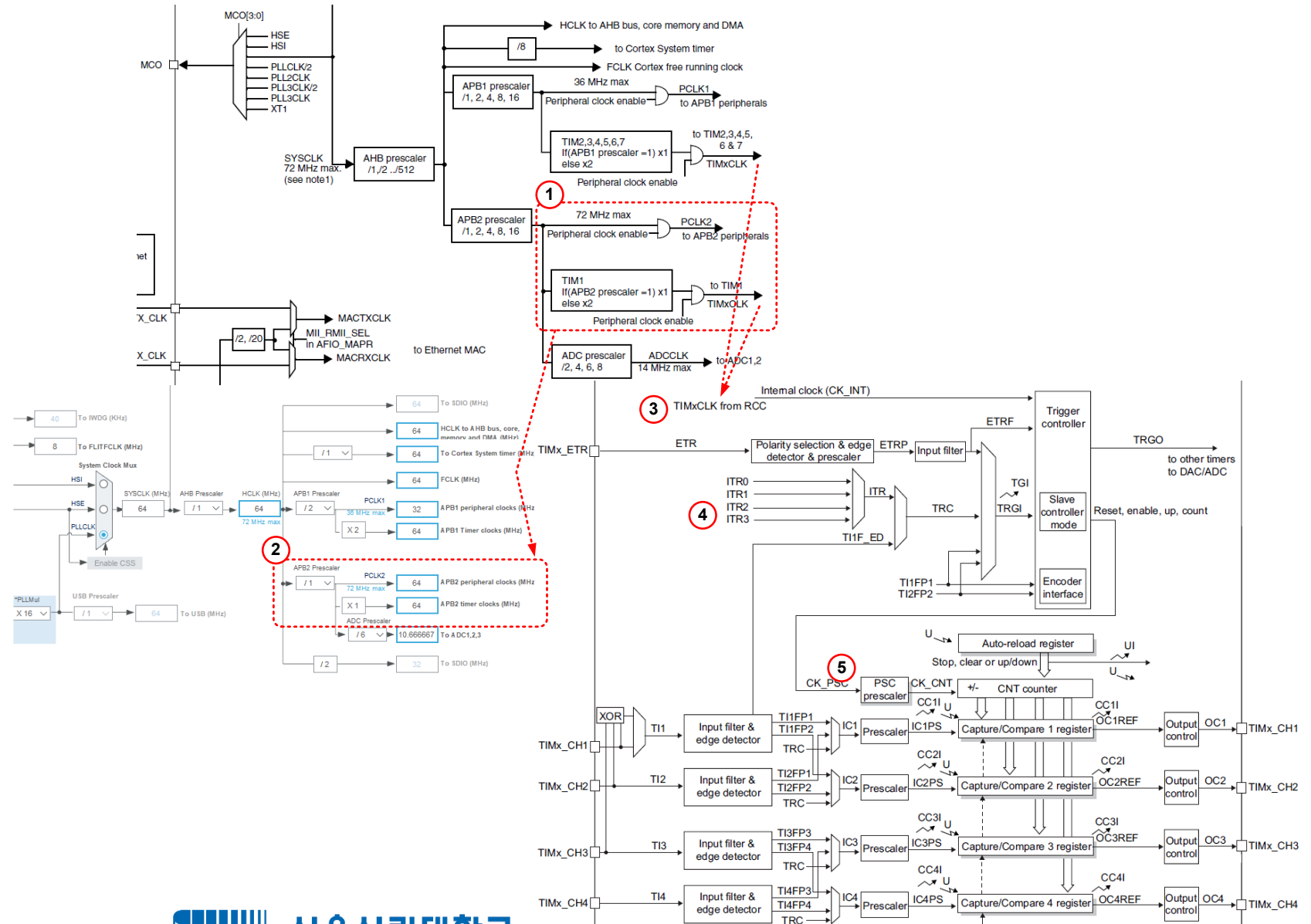
✓ STM32F103 내부 구성도 :

- ③번과 ①번에서 보여준 것과 같이 advanced control timer TIM1과 TIM8은 core clock과 동일한 최대 72[MHz]인 APB2에 연결되어 있다.
- ②번 부분은 CubeMX의 Clock Configuration tab에 있는 APB1 timer clock을 설정하는 부분.
- APB1 clock은 ⑤번 basic timer들과 ④번 general purpose timer들을 위한 clock으로 사용된다.

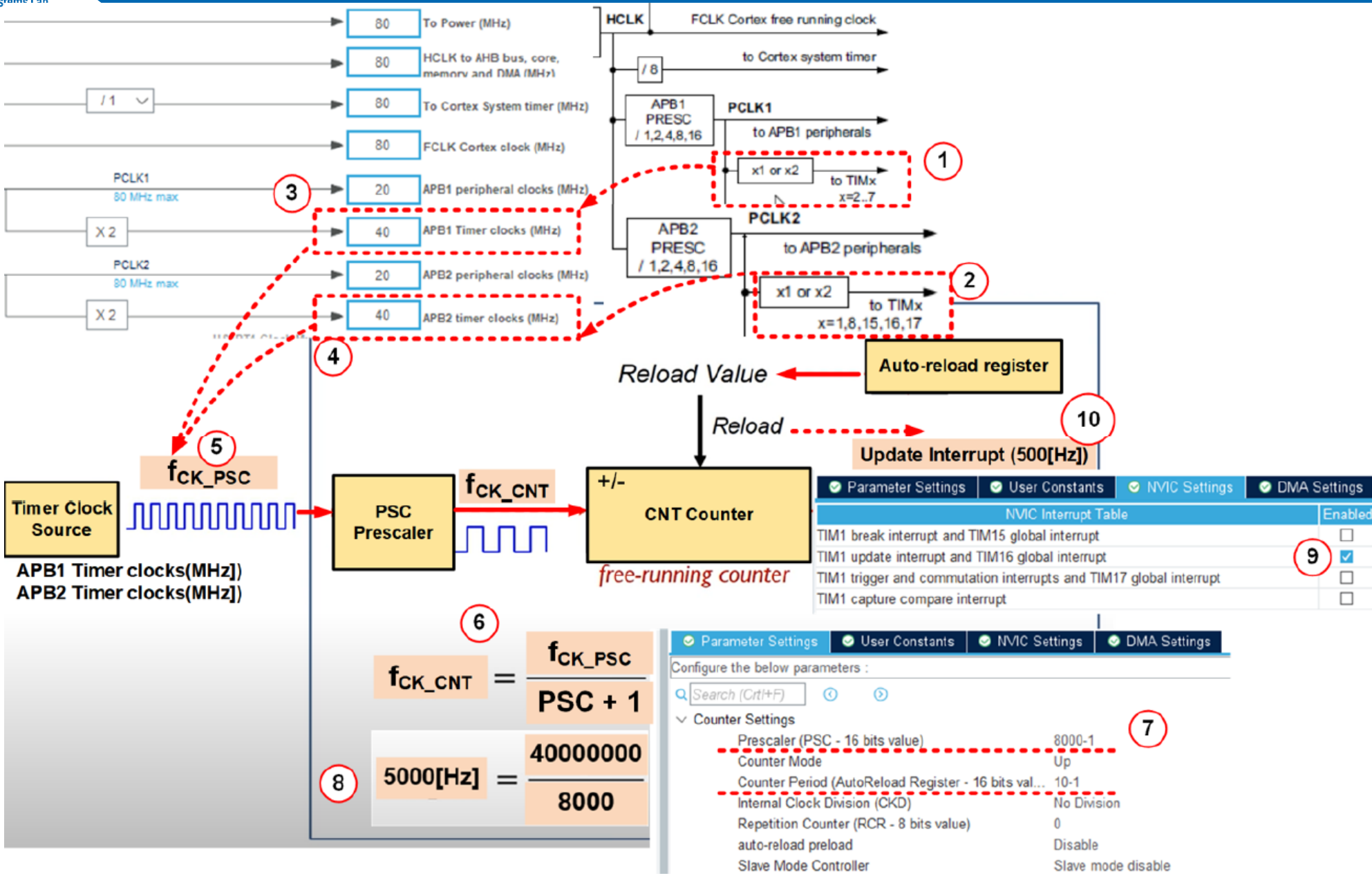
Timer 전체 구성도.

✓ ⑤번에서 보여준 PSC prescaler, CNT counter, ARR Auto-Reload Register. 이들 register의 값은 counter가 동작하는 상황에서도 읽을 수 있으며, 새로운 값을 설정할 수도 있다.

✓ HAL library에서 TIM_TypeDef 구조체에 각각 PSC, CNT, ARR 멤버 변수로 선언되어 있으므로 TIMx->PSC와 같이 접근하면 된다.



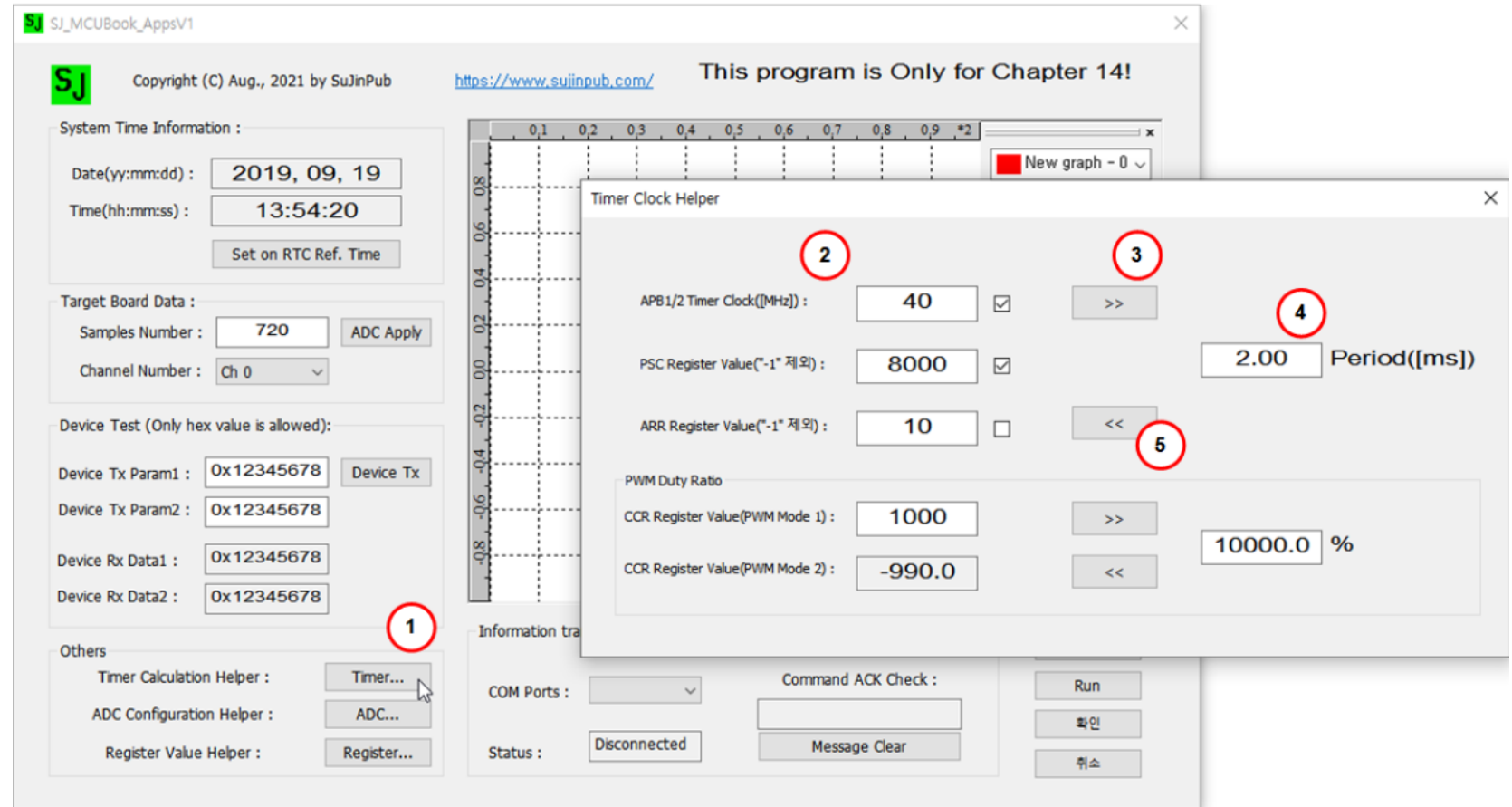
CubeMX와 Timer 설정 방법 - 1



- ①번처럼 APB1에 TIM2~TIM7까지 연결되어 있다.
- ②번처럼 TIM1, TIM8 그리고, TIMx에 해당하는 TIM15, TIM16, TIM17이 APB2에 연결된 것을 볼 수 있다.
- ①번과 ②번은 각각 CubeMX의 Clock Configuration tab에서 ③번과 ④번에 해당한다.
- 이들은 ⑤번의 f_{CK_PSC} 에 해당하며, timer 내부적으로 ⑥번과 같은 관계를 가진다.
- ⑦번과 같이 CubeMX에서 Prescaler(PSC-16 bits value)에 “8000-1”이라고 설정한다면, ⑧번처럼 $f_{CK_CNT} = 5000[Hz]$ 가 될 것이다.
- 16bits ARR(Auto-reload register)에 $10-1=9$ 를 지정해 주면, CNT Counter는 지정한 Counter Mode가 Up이므로 “0”부터 “9”까지 10번 count하고 UI(Update Interrupt)를 발생시키게 된다.
- 결국, 500[Hz](즉, 2[ms])마다 Update Interrupt를 반복적으로 발생시킨다.

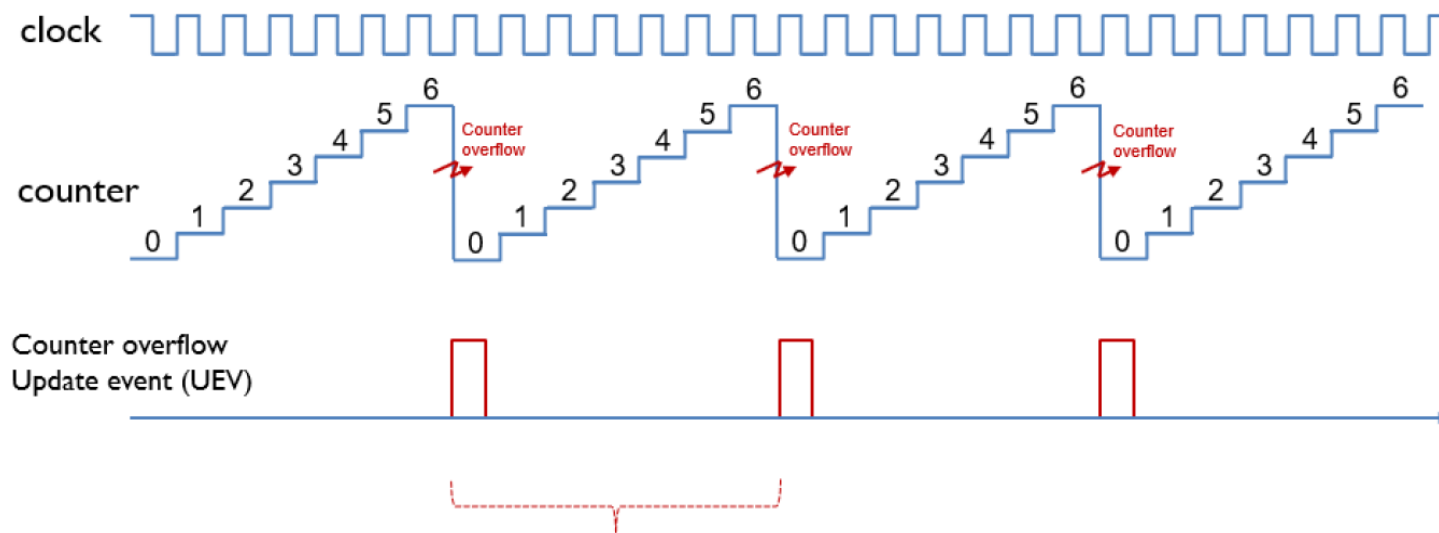
CubeMX와 Timer 설정 방법 - 3

- ✓ Timer 1을 이용하여 500[Hz] (즉, 2[ms]) 마다 UI를 발생시키고 싶다면,
 $CK_PSC=40[MHz]$ 이므로 $40000000/500 = (Prescaler\text{설정값}+1) \times (Counter\text{Period}+1)$ 이 된다.



✓ ARR=6, RCR=0에서 Up Counting의 경우 :

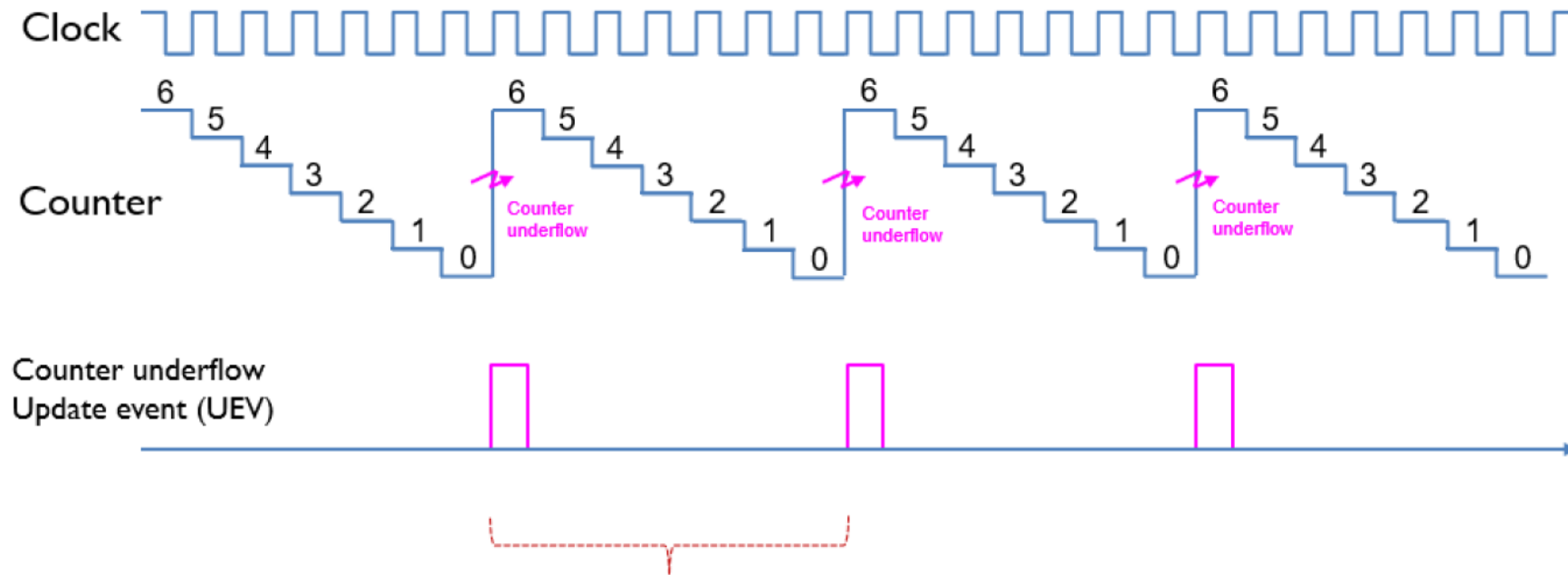
- CNT=0부터 시작하여 CNT=ARR(여기서는 6)값까지 증가한 후에 다시 0부터 counting을 시작하는 작업을 계속해서 반복하는데, CNT=0이 될 때 마다 overflow가 발생하고, update event(UEV)와 UI(즉, Update Interrupt)가 발생한다.
- 원하는 ARR개의 count 값을 반영하기 위해서는 0부터 시작하므로 ARR-1을 지정해 주어야 한다.



$$\begin{aligned} \text{Period} &= (1 + \text{ARR}) * \text{Clock Period} \\ &= 7 * \text{Clock Period} \end{aligned}$$

✓ ARR=6, RCR=0에서 Down Counting의 경우 :

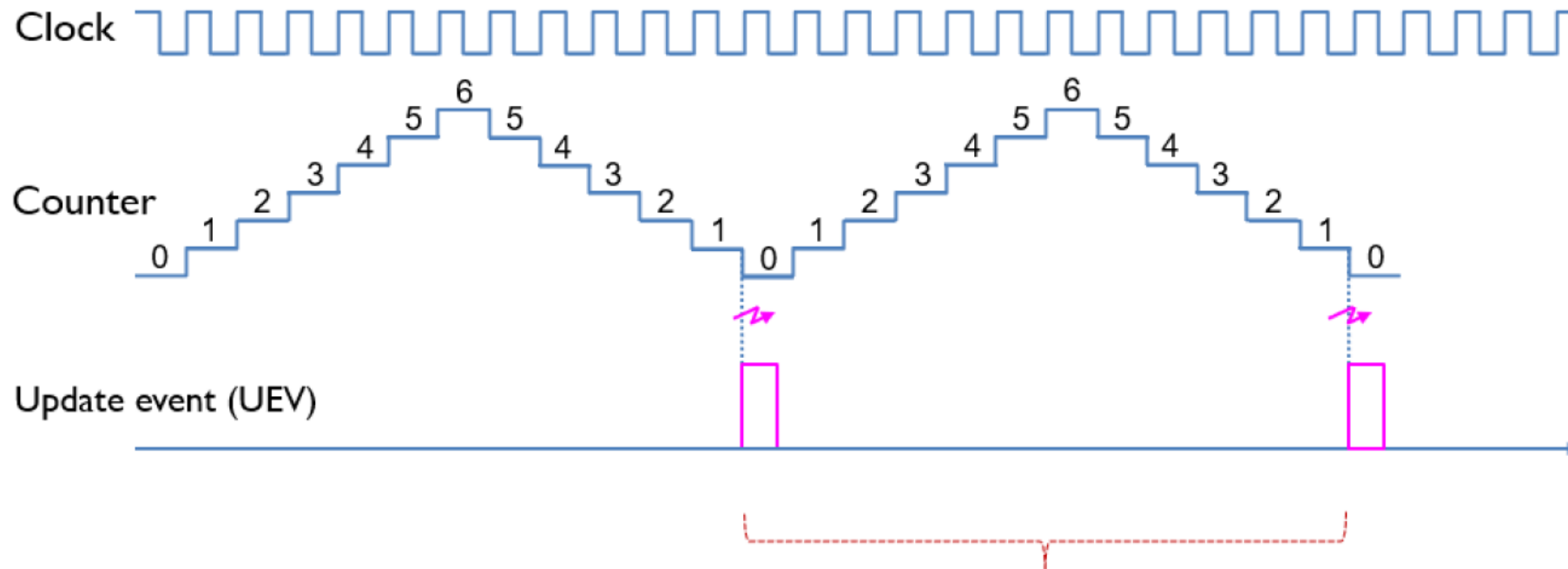
- 원하는 ARR개의 count 값을 반영하기 위해서는 ARR-1을 지정



$$\begin{aligned} \text{Period} &= (1 + \text{ARR}) * \text{Clock Period} \\ &= 7 * \text{Clock Period} \end{aligned}$$

✓ ARR=12, RCR=0에서 Center-aligned Counting의 경우 :

- Up 또는 Down counting mode에서는 ARR=6-1을 지정해 주어야 하고, Center-aligned mode에서는 $ARR=6 \times 2 = 12$ 를 지정하면, 정확히 2배의 크기가 된다.



$$\begin{aligned} \text{Period} &= (1+ARR) * \text{Clock Period} \\ &= 13 * \text{Clock Period} \end{aligned}$$

✓ 3개 timer들을 각각 다음과 같은 사양으로 생성하고자 한다.

1) TIM1 :

- APB2 timer clock = 36[MHz], timer 동작 주파수 = 1[ms], Up Counting mode
- 목 적 : 전체 application의 기준 시간으로 사용.

2) TIM2 :

- APB1 timer clock = 18[MHz], timer 동작 주파수 = 2[ms], Center aligned mode
- 목 적 : application을 구성하는 일정 함수의 기준 시간으로 사용.

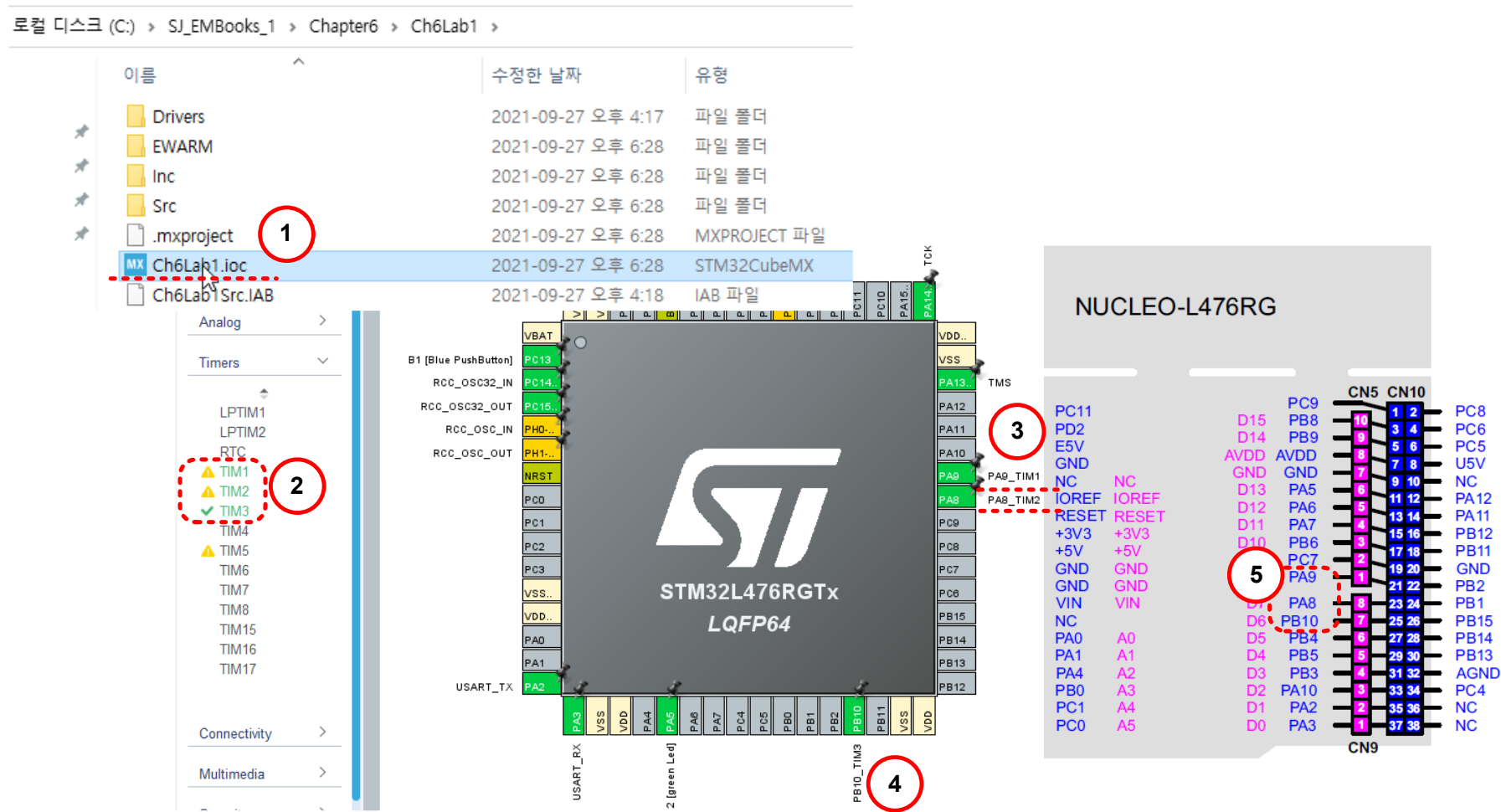
3) TIM3 :

- APB1 timer clock = 18[MHz], timer 동작 주파수 = 100[ms],
- 목 적 : 500[ms] 주기로 PC에 데이터를 전송.

3) TIM3 :

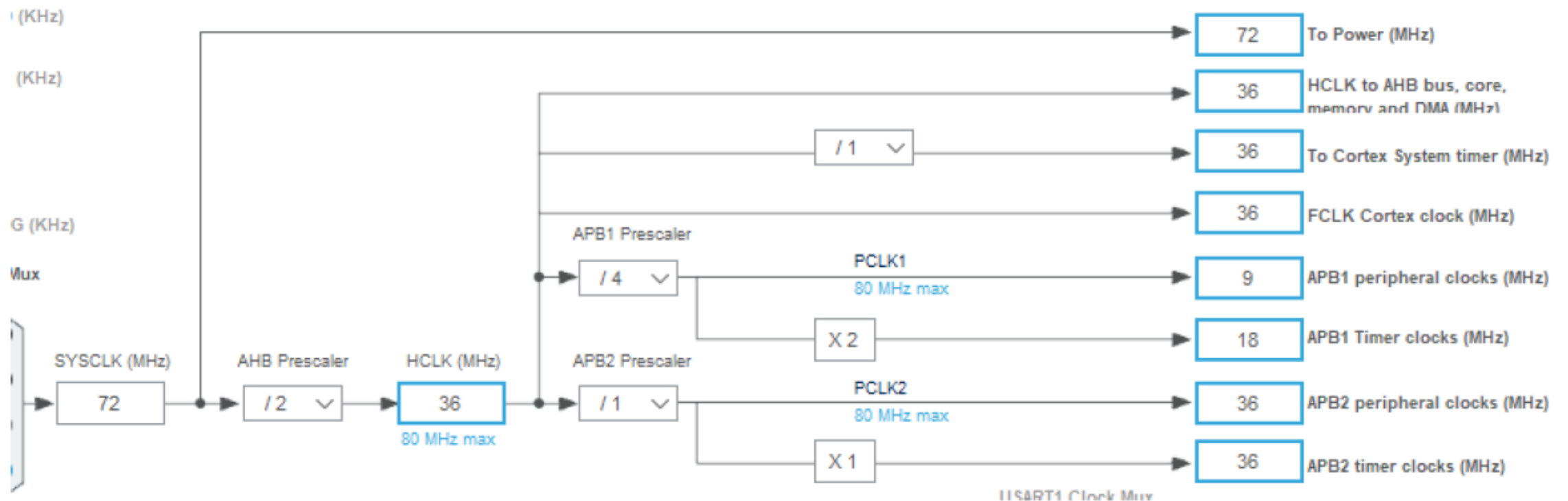
- APB1 timer clock = 18[MHz], timer 동작 주파수 = 100[ms],
- 목 적 : 500[ms] 주기로 PC에 데이터를 전송.
- UART 통신으로 PC에 연결하고, baud rate로 115200[bps]를 사용할 것이다. 그러면, 1byte 전송에 대략 90[us]가 소요된다. 한 번에 보낼 전체 데이터 block의 크기가 $28 \times 2[\text{bytes}] = 56[\text{bytes}]$ 이면, 대략 $56 \times 90[\text{us}] = 5[\text{ms}]$ 보다 약간 많은 시간이 필요하다. 그러나, 500[ms] 마다 PC로 데이터를 전송할 것이므로 전송상에 충돌이 발생할 일이 없다.
- 그 밖의 구현 목표 : timer 1의 UI가 발생하면, PA9 pin이 toggle하고, timer 2의 UI는 PA8 pin, 그리고, timer 3의 UI는 PB10 pin을 toggling 하도록 coding 할 것.

- ✓ Timer 3개 설정, 각각의 timer UI(Update Interrupt)를 위해 ⑤번과 같이 GPIO output port 설정!



✓ Timer1, Timer2, 그리고, Timer3 요구 사항에 따른 Clock Configuration 설정 :

- Timer1을 위한 APB2 timer clock은 36[MHz].
- Timer2와 Timer 3을 위한 APB1 Timer clock은 18[MHz]



✓ Timer 1 Interrupt Service Routine 생성 방법

6 Clock Source: Internal Clock

7 Counter Settings: Prescaler (PSC - 16 bits value) 6000-1, Counter Mode Up, Counter Period (AutoReload Register - 16 bits value) 6-1, Internal Clock Division (CKD) No Division, Repetition Counter (RCR - 8 bits value) 0, auto-reload preload Disable.

8 NVIC Interrupt Table:

Interrupt	Enabled	Preemption Priority	Sub Priority
TIM1 break interrupt and TIM15 global interrupt	☐	0	0
TIM1 update interrupt and TIM16 global interrupt	☒	0	0
TIM1 trigger and commutation interrupts and TIM17 global interrupt	☐	0	0
TIM1 capture compare interrupt	☐	0	0

9 File Selection: `Stm324xx_it.c`

10 Function Declaration:

```
void TIM1_UP_TIM16_IRQHandler(void)
/* USER CODE BEGIN TIM1_UP_TIM16_IRQn 0 */
/* USER CODE END TIM1_UP_TIM16_IRQn 0 */
HAL_TIM_IRQHandler(&htim1);
/* USER CODE BEGIN TIM1_UP_TIM16_IRQn 1 */
/* USER CODE END TIM1_UP_TIM16_IRQn 1 */
/*
 * @brief This function handles TIM1 update interrupt and
 */

```

11 Function Implementation:

```
void HAL_TIM_IRQHandler(TIM_HandleTypeDef *htim)
/* Capture compare 1 event */
if ( __HAL_TIM_GET_FLAG(htim, TIM_FLAG_CC1) != RESET)
{
    if ( __HAL_TIM_GET_IT_SOURCE(htim, TIM_IT_CC1) != RESET)
    {

```

- ✓ 제일 먼저 ⑥번과 같이 Timer의 Clock source를 설정해 준다.
- ✓ ⑦번과 같이 Timer에 대한 parameter들을 설정해 준다.
 - TIM1의 경우, APB2=36[MHz], UI=1[ms] 이므로 PSC=6-1, ARR=6000-1이 된다.
- ✓ 같은 방식으로 Timer 2와 Timer 3도 설정!
- ✓ 3개의 Timer들에 대한 모든 설정을 완료한다.

The screenshot shows the 'Timer Clock Helper' dialog box with the following settings:

- APB1/2 Timer Clock([MHz]) : 36 ☒ >>
- PSC Register Value("1" 제외) : 6 ☒ 1 Period([ms])
- ARR Register Value("1" 제외) : 6000.0 ☐ <<
- PWM Duty Ratio
 - CCR Register Value(PWM Mode 1) : 1000 >> 16.7 %
 - CCR Register Value(PWM Mode 2) : 4998.0 <<

- ✓ ⑧번과 같이 UI가 발생하면 처리할 ISR관련 code를 생성하도록 NVIC Settings에서 TIM1 update interrupt and TIM_16 global interrupt를 check해 준다.
 - Timer 2와 Timer 3도 동일하게 설정!
- ✓ GENERATE CODE button을 click하여 C framework 파일들을 생성하면, ⑨번과 같이 stm32l4xx_it.c file안에 ⑩번과 같이 timer1의 Update Interrupt가 발생하였을 때에 처리할 Interrupt Service handler인 TIM1_UP_TIM16_IRQHandler() 함수 관련 code가 추가된다.
- ✓ 이 함수는 내부적으로 ⑪번과 같이 HAL_TIM_IRQHandler() 함수를 호출하는 것을 볼 수 있다.
- ✓ 이 함수의 내용을 살펴 보면, Capture compare 1 event부터 4 event까지 처리하고, 이후에 ⑧번에서 check한 TIM Update event를 처리하는데, 구체적으로 HAL_TIM_PeriodElapsedCallback() 함수가 처리하도록 되어 있다.

- ✓ 함수는 `__weak` 함수로 되어 있으므로, `main.c` 파일에서 새롭게 `__weak` keyword 를 제외한 동일한 함수 prototype을 갖는 `HAL_TIM_PeriodElapsedCallback()` 함수를 정의해 준다.
- ✓ 이제부터는 Timer1 UI가 발생하면, `__weak` keyword가 빠진 지금 작성한 `HAL_TIM_PeriodElapsedCallback()` 함수가 호출되고, 이 함수의 입력 매개변수 `TIM_HandleTypeDef *htim`의 Instance member의 값이 TIM1이면, timer 1 UI가 발생한 것을 의미한다.
- ✓ 동일한 방식으로 TIM2이면, timer 2 UI가 발생한 것을 의미하고, TIM3이면, timer 3 UI가 발생한 것을 의미한다.
- ✓ 이것을 반영하여 `HAL_TIM_PeriodElapsedCallback()` 함수를 coding한다.

✓ 다음과 같이 Timer callback 함수 Coding :

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {  
    if(htim->Instance == TIM1) { // Whenever TIM1 UI is happened every 1[ms]  
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_9);  
    }  
    if(htim->Instance == TIM2) { // Whenever TIM2 UI is happened every 2[ms]  
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_8);  
    }  
    if(htim->Instance == TIM3) { // Whenever TIM3 UI is happened every 100[ms]  
        HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_10);  
    }  
}
```

- ✓ 이와 같은 Timer Interrupt가 발생하기 위해서는 main while-loop에 들어가기 전에 반드시 HAL_TIM_Base_Start_IT() 함수를 각각의 timer 마다 다음과 같이 호출해 주어야 한다.

```
HAL_TIM_Base_Start_IT(&htim1);  
HAL_TIM_Base_Start_IT(&htim2);  
HAL_TIM_Base_Start_IT(&htim3);
```

✓ Timer2와 Timer3에 대한 CubeMX 설정 :

TIM2 Mode and Configuration

Mode

Slave Mode

Trigger Source

Clock Source

Channel1

Channel2

Channel3

Channel4

Combined Channels

Configuration

☒ Parameter Settings
 ☒ User Constants
 ☒ NVIC Settings
 ☒ DMA Settings

Configure the below parameters :

Search (Ctrl+F)

12

Counter Settings

Prescaler (PSC - 16 bits value) 3000-1

Counter Mode Center Aligned mode3

Counter Period (AutoReload Register - 32 bits val... 12

Internal Clock Division (CKD) No Division

auto-reload preload Disable

TIM3 Mode and Configuration

Mode

Slave Mode

Trigger Source

Clock Source

Channel1

Channel2

Channel3

Channel4

Combined Channels

Configuration

☒ Parameter Settings
 ☒ User Constants
 ☒ NVIC Settings
 ☒ DMA Setting

Configure the below parameters :

Search (Ctrl+F)

13

Counter Settings

Prescaler (PSC - 16 bits value) 30000-1

Counter Mode Up

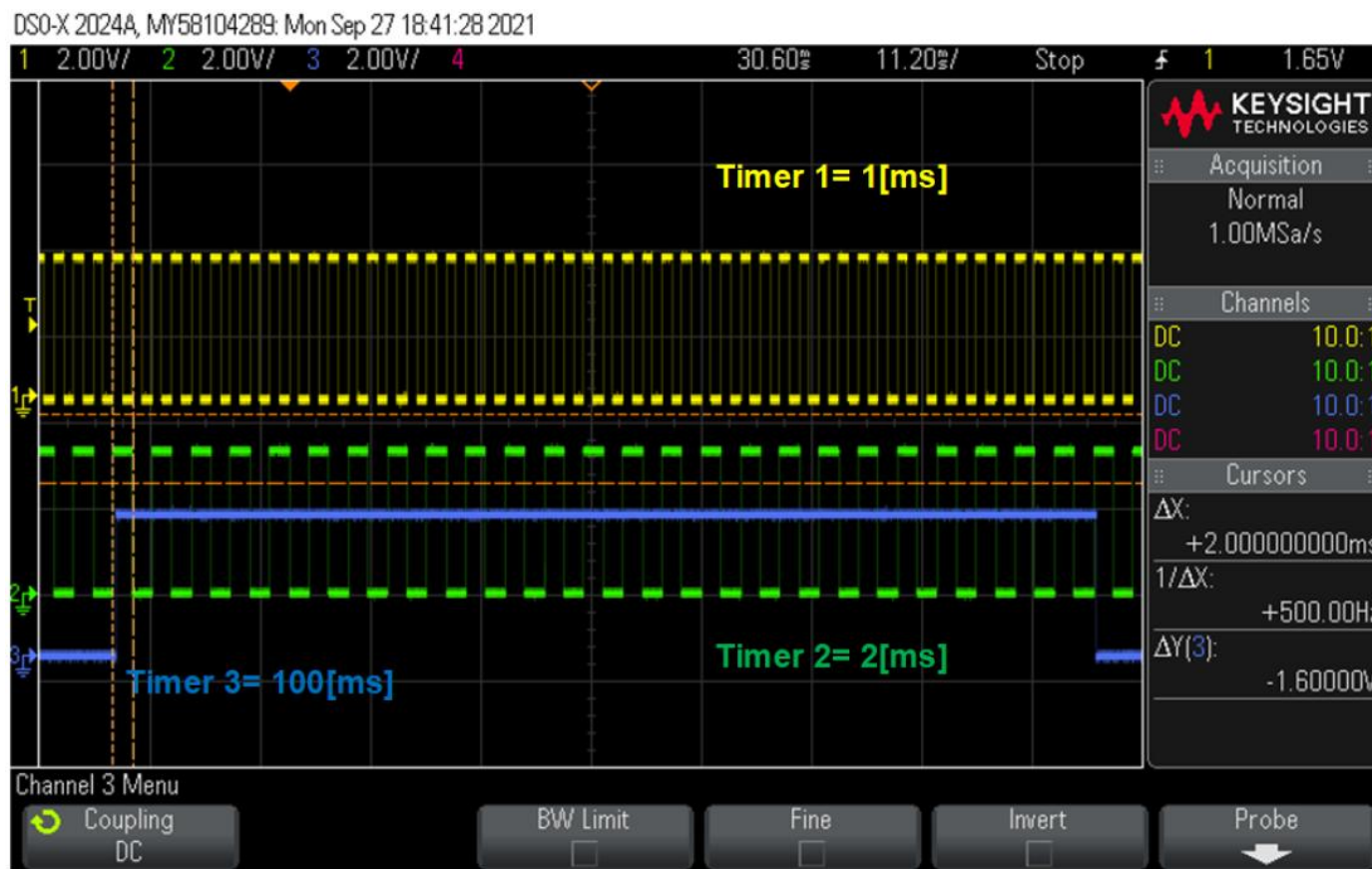
Counter Period (AutoReload Regist... 60-1

Internal Clock Division (CKD) No Division

auto-reload preload Disable

Timer 관련 예제 결과 확인과 분석 - 1.

- ✓ Timer1, Timer2, 그리고, Timer3에 따른 각각의 timer UI에 연결된 GPIO output 출력 결과!



✓ 결과 확인 방법 :

- 다음과 같이 3개의 breakpoint들을 설정하고 모두 멈추는 것 확인.

Project - IAR Embedded Workbench IDE - Arm 8.50.1

File Edit View Project Debug Disassembly ST-Link Tools Window Help

Workspace

Ch6Lab4

Files

- Ch6Lab4 - Ch6Lab4
 - Application
 - EWARM
 - User
 - main.c
 - stm32f1xx_hal_m...
 - stm32f1xx_it.c
 - Drivers
 - Output

main.c

```

HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *)
374 HAL_NVIC_EnableIRQ(EXTI15_10_IRQn);
375
376 }
377
378 /* USER CODE BEGIN 4 */
379 void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
380 {
381     if(htim->Instance == TIM1) { // Whenever TIM1 UI is happened every 1[ms]
382         HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_9);
383     }
384
385     if(htim->Instance == TIM2) { // Whenever TIM2 UI is happened every 2[ms]
386         HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_8);
387     }
388     if(htim->Instance == TIM3) { // Whenever TIM3 UI is happened every 100[ms]
389         HAL_GPIO_TogglePin(GPIOB, GPIO_PIN_10);
390     }
391 }
392 /* USER CODE END 4 */
393
394 /**

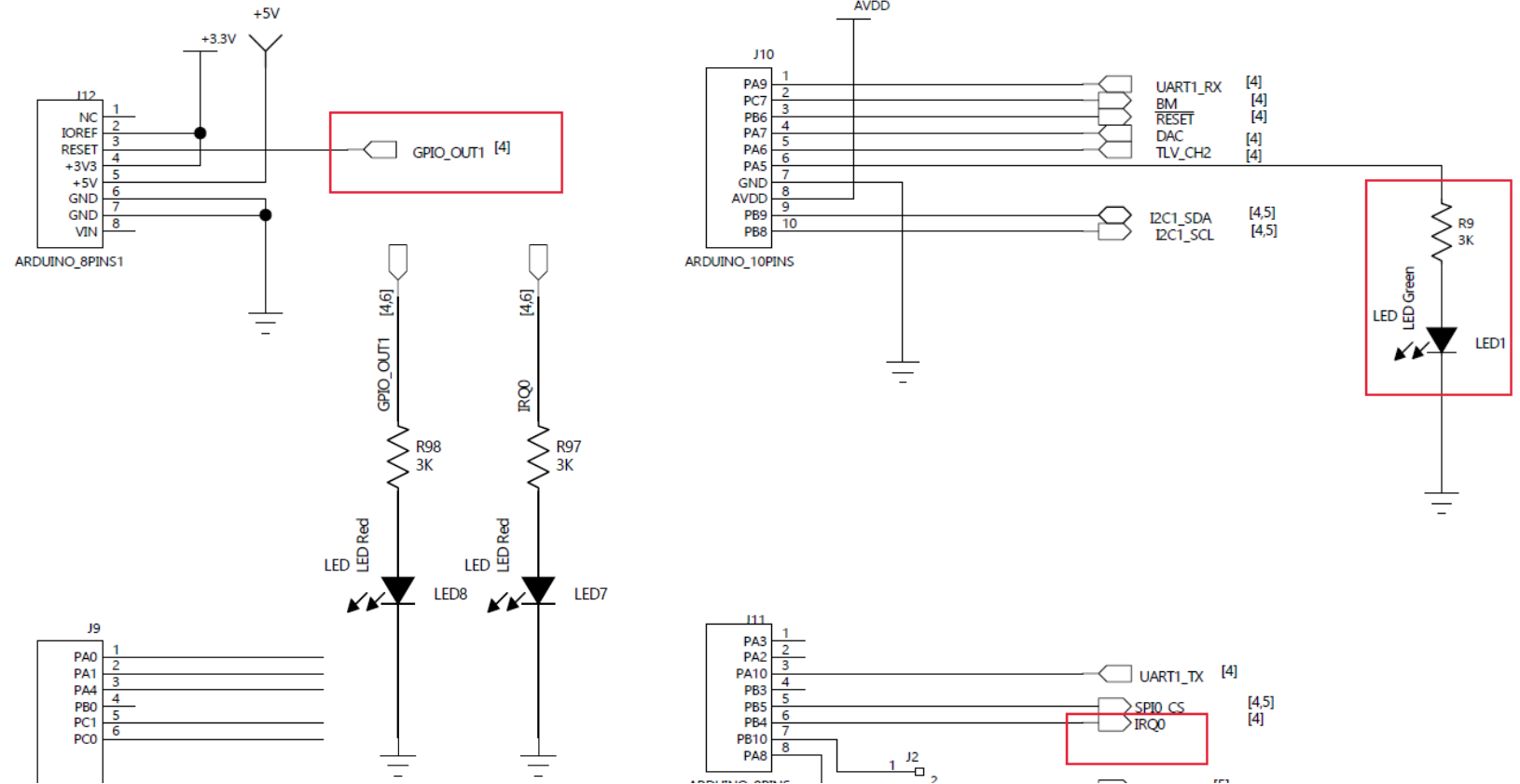
```

Breakpoints

	Type	Location	Extra
✓	Code	main.c:389.5.	type: default (auto)
✓	Code	main.c:386.5.	type: default (auto)
✓	Code	main.c:382.5.	type: default (auto)

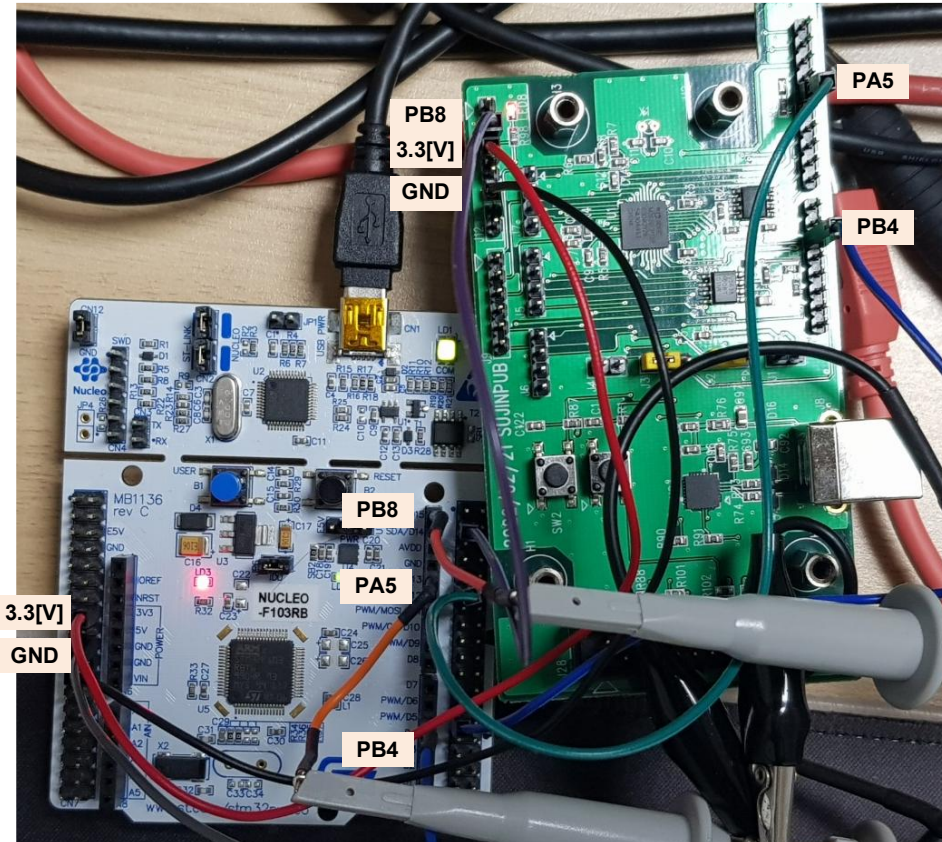
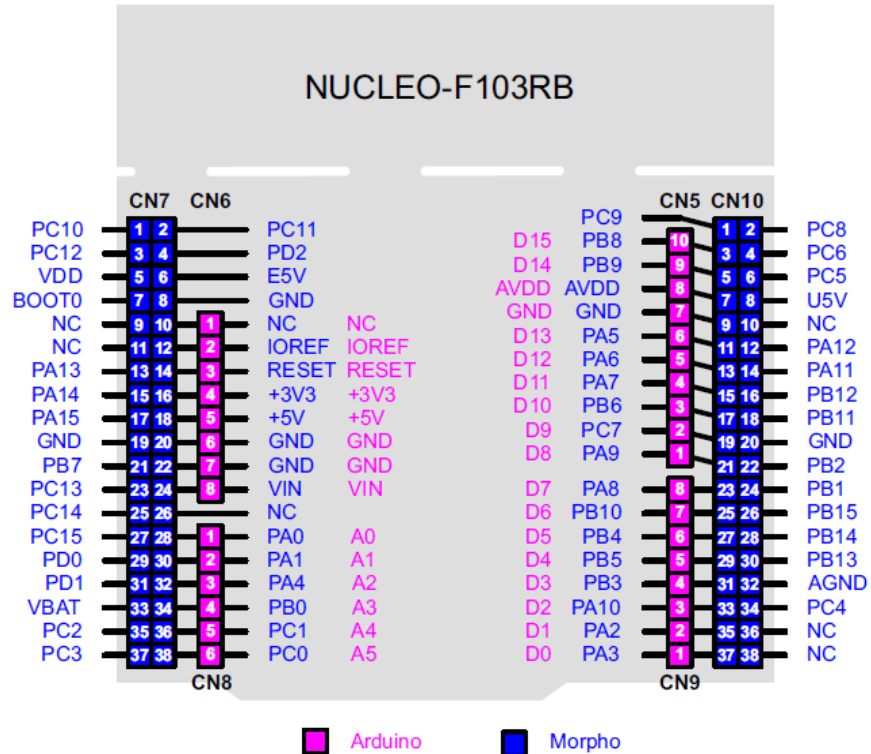
✓ 코드 수정

- SJ_MCUBook_M3 보드에 있는 LED1, LED7, 그리고, LED8을 각각 Timer 1, Timer 2, 그리고, Timer 3에 연결
- LED8번의 경우, RESET pin에 연결되어 있으므로 Nucleo 보드와 SJ_MCUBook_M3 보드는 서로 분리하여 제공하는 5개의 Jumper wire로 연결.
- 3.3[V], GND, PA5, PB4, PB8 pin을 이용.



✓ 코드 수정

▪ 연결 사진



- ✓ 각각의 Timer를 위한 Counter 변수 사용.
 - 전역 변수로 Tim1Cnt, Tim2Cnt, 그리고, Tim3Cnt을 정의하여 각각의 LED 점멸 주기 조정하는 코드 작성
(조교 확인필수)

✓ Watchdog Timer :

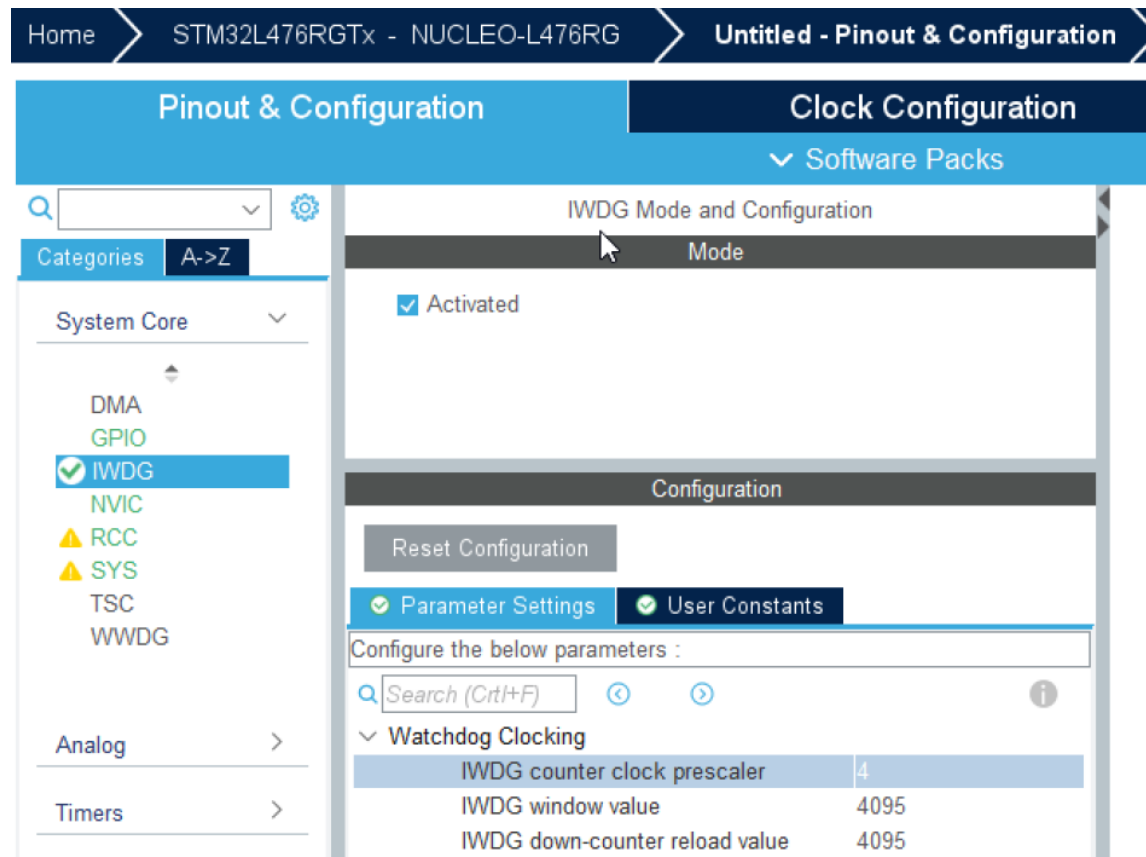
- System 고장 방지를 위한 system 감시 timer로서 전원인가 시에 enable 상태로 되며, 주기적으로 system reset을 발생한다. 그러므로 core가 살면, 제일 먼저, watchdog timer를 disable해 주어야 한다.
- MCU가 제공하는 임의의 입/출력 주변 장치를 통해서 데이터가 계속해서 잘못 들어오거나 또는 뭔가 잘못되어 임의의 반복문에서 빠져나오지 못할 때에 system을 reset 시켜서 처음부터 다시 실행하도록 한다.
- 임의의 time period 동안 timer를 clear하지 않으면, 즉, time out이 되면, 전체 system을 reset하도록 하는 특별한 목적의 timer를 watchdog timer라고 한다.

✓ STM32 MCU는 2가지 종류의 watchdog timer를 제공 :

- 1) Independent watchdog(IWDG) : MCU main clock과 상관없이 동작.
- 2) Window watchdog(WWDG) : APB clock으로 동작.

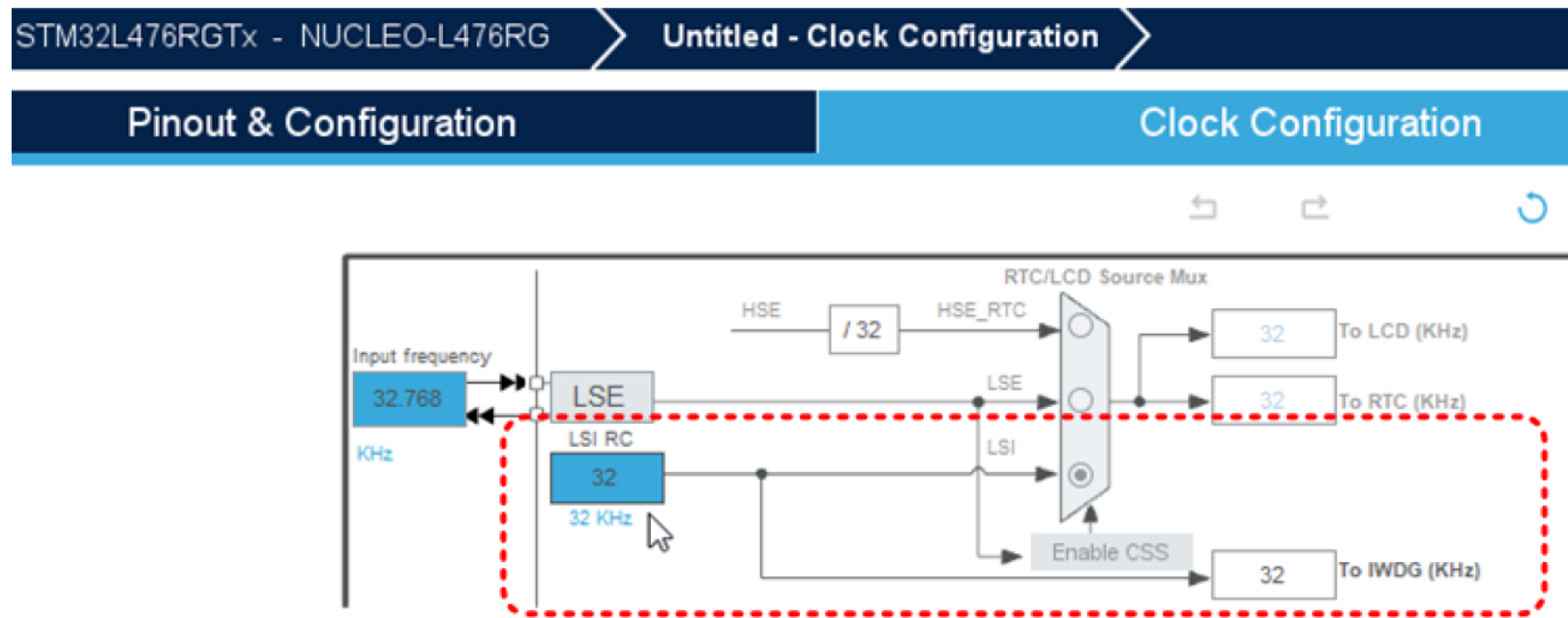
✓ Watchdog Timer 사용 방법 :

- 우선, 사용할 watchdog를 Activated checkbox를 선택! (F103에는 window 설정 없음)



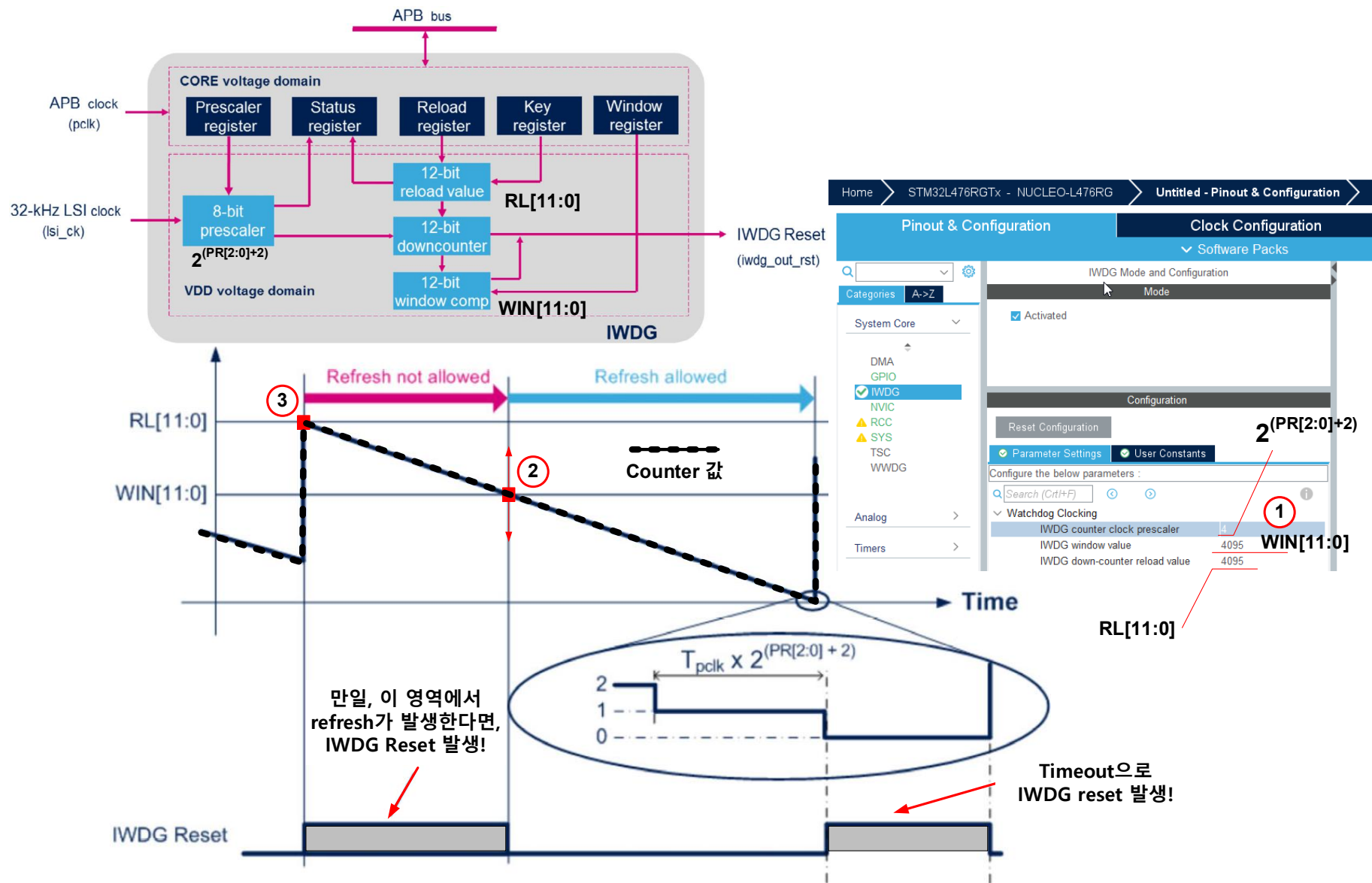
✓ Watchdog Timer 사용 방법 :

- IWDG는 MCU 내부의 32[kHz] 전용 RC clock을 사용.
- main clock과 독립적인 clock을 사용하는데, 문제는 이 clock이 정확하지 않다는 것이다. 즉, 내부 RC clock을 사용하므로 온도에 따라서 그 특성이 달라질 수 있다. 즉, 설정한 시간 구간에 대한 오차가 크다는 의미.
- STM32F103의 경우, 32[kHz]가 아닌 40[kHz]!



[그림 6.3-3] IWDG을 위한 RC clock.

Watchdog Timer 개념과 사용 방법 - 4.



- STM32L476의 경우, LSI RC는 32[kHz]이고, STM32F103과 STM32F302는 모두 LSI RC가 40[kHz]인데 주의하자.
- 입력으로 주어진 LSI RC 32[kHz]를 8bits prescaler로 낮추어 주고, 이 clock으로 12bits reload register에 설정한 개수만큼 counting한다는 의미.
- 12bits WIN[11:0]에 설정한 값보다 작은 구간에서 이 counting 값을 다시 reload하면 IWDG reset이 발생하지 않고, 그렇지 않으면 발생한다는 의미이다.
- 현재, WIN[11:0] register의 값은 default인 0x0FFF=4095이다. 그리고, RL[11:0]의 값도 4095이다. 그러면, ②번의 점이 위로 이동하여 ③번으로 이동하여 하나의 점이 되고, 이것은 결국, Refresh not allowed 구간이 제거 된다.
- 결국, Refresh allowed 구간만 존재하게 되어 임의의 시간 구간에서 RL 값을 refresh해도 IWDG reset이 발생하지 않는다는 의미가 된다.

- t_{ms} [ms] 동안 시간 구간을 설정하여 이 시간 구간 안에 RL 값을 refresh하지 않으면, IWDG reset이 걸리도록 하고 싶은 경우에 RL[11:0]에 설정할 값은 다음 수식을 이용하면 된다.

$$RL[11:0] = (t_{ms} \times 40000) / (2^{PR[2:0]+2} \times 1000) - 1 \quad (\text{STM32F103과 STM32F302의 경우})$$

- main() 함수에서 MX_IWDG_Init() 함수를 호출하여 일단, IWDG이 enable되면, 이후부터는 disable되지 않는다.
- Refresh 함수 : HAL_IWDG_Refresh(&hiwdg);
- 이제, 10[ms] 시간 구간 동안 RL이 refresh 되지 않으면 자동으로 IWDG reset이 걸리도록 CubeMX에 RL[11:0]의 값을 계산하면 다음과 같다.
 - PR = 4로 선택, 분주비는 $2^{PR[2:0]+2} = 2^6 = 64$, 따라서 40kHz 의 1/64배
 - $RL[11:0] = (10 \times 40000) / (2^6 \times 1000) - 1 = 5.25$ 가 되며 5 (9.6ms) 또는 6 (11.2ms)으로 선택하면 됨. .

✓ Case 1: Refresh 없어서 timeout 및 watchdog reset 실습

- main.c에 다음 추가
- 정상부팅 되는 경우는 1번 깜빡이고, watchdog에 의해서 reset되면 3번 0.1초씩 세번 연속 깜빡임 (조교확인필수)

```
.00 /* Initialize all configured peripherals */
.01 MX_GPIO_Init();
.02 MX_USART2_UART_Init();
.03 MX_TIM1_Init();
.04 MX_TIM2_Init();
.05 MX_TIM3_Init();
.06 MX_IWDG_Init();
.07 /* USER CODE BEGIN 2 */
.08 ShowResetCause();
.09
.10 //uint32_t last = HAL_GetTick();
.11 //const uint32_t kick_ms = 1000; // 타임아웃보다 충분히 짧게(예: 1 s)
.12
.13 /* USER CODE END 2 */
.14
.15 /* Infinite loop */
.16 /* USER CODE BEGIN WHILE */
.17 for (int i=0;i<5;i++){ Blink(1,500); }
.18
.19 while (1)
.20 {
.21     /* USER CODE END WHILE */
.22
.23     /* USER CODE BEGIN 3 */
.24
.25     /* USER CODE END 3 */
.26 }
```

```
/* USER CODE BEGIN 4 */
static void Blink(uint8_t n, uint32_t ms) {
    for (uint8_t i=0;i<n;i++){
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
        HAL_Delay(ms);
        HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
        HAL_Delay(ms);
    }
}

static void ShowResetCause(void){
    if (__HAL_RCC_GET_FLAG(RCC_FLAG_IWDGRST)) Blink(3,100); // IWDG 리셋: LED 3번
    else Blink(1,200); // 일반 부팅: LED 1번
    __HAL_RCC_CLEAR_RESET_FLAGS();
}
/* USER CODE END 4 */
```

✓ Case 2: 주기적인 refresh 통해서 mcu 정상동작하면 reset 방지 (조교확인필수)

- main.c 다음과 같이 수정

```

107  /* USER CODE BEGIN 2 */
108  ShowResetCause();
109
110  uint32_t last = HAL_GetTick();
111  const uint32_t kick_ms = 1000; // 타임아웃보다 충분히 짧게(예: 1 s)
112
113  /* USER CODE END 2 */
114
115  /* Infinite loop */
116  /* USER CODE BEGIN WHILE */
117  //for (int i=0;i<5;i++){ Blink(1,300); }
118
119  while (1)
120  {
121      /* USER CODE END WHILE */
122
123      /* USER CODE BEGIN 3 */
124      Blink(1,500);
125
126      // 주기적 refresh
127      if (HAL_GetTick() - last >= kick_ms) {
128          HAL_IWDG_Refresh(&hiwdg); // F1은 언제나 OK, L4/H7은 윈도우 사용 시 시간 맞춰야 함
129          last += kick_ms;
130      }
131
132      // 버튼을 누르면 '일부러' refresh 중단해서 리셋을 유발
133      if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) == GPIO_PIN_RESET) {
134          while(1) { /* 버튼 동안 멈춤 → 곧 리셋 */ }
135      }
136  }
137  /* USER CODE END 3 */
138  }

```